

Warum ist das Laravel-Actions-Pattern hinsichtlich der Skalierbarkeit der App vorteilhaft?

Grundsätzlich dient das Actions-Pattern einer verbesserten Erweiterbarkeit und Wartbarkeit der App. Indem jede Action-Klasse nur EINE eigene Verantwortlichkeit bzw. Aufgabe für die Funktionalität der App besitzt, kann diese Verantwortlichkeit genau mithilfe der zugehörigen Klasse getestet oder erweitert werden. Zu einer Route kann direkt eine Action-Klasse zugeordnet werden, sodass Controller optional werden. Actionklassen können aber auch interne Business-Logik ohne zugehörige Route ausführen, was dann nur vom Backend genutzt wird. Ein immenser Vorteil ist vor Allem die Möglichkeit, Actions bzw. deren Funktionalität direkt per Instanziierung in Livewire-Components integrieren zu können, ohne eine Controller-Instanz referenzieren zu müssen.

Im Gegensatz dazu sind mit Verantwortlichkeiten „überfüllte“ Controller-Klassen unübersichtlich und erfüllen nicht das Prinzip „lose Kopplung und getrennte Verantwortlichkeiten“ von Klassen oder Modulen, was Erweiterbarkeit und Wartbarkeit verkompliziert. Werden Action-Klassen genutzt, bleibt die Business-Logik allein in diesen Klassen, und die Controller können andere Aufgaben in großen Apps erfüllen, etwa die Workflow-Orchestrierung oder Details des HTTP-Request/Response-Handlings festlegen.

Hinsichtlich der Skalierbarkeit ermöglicht das Pattern, die Klasseninstanzen bzw. Actions auch als Laravel-Jobs in Queues laufen zu lassen, sodass eine geschickte Parallelprozessierung durch verschiedene Worker in einer separaten Redis-Job-Queue (mit externem Redis-Server) möglich ist. So können die Actions auf dem Redis-Queue-Server für mehrere parallel laufende Web-Server genutzt werden (in einer Multi-Server-Architektur etwa). Bei wachsender Nutzlast kann derselbe Code der Actions bzw. Business-Logik also unabhängig von der erweiterten Infrastruktur wiederverwendet werden.

Weiterhin sind die Action-Klassen direkt für die Implementierung von externen Microservices nutzbar, was die Migration einer App bzw. die schrittweise Extraktion einzelner Services in eine Service-orientierte Architektur enorm erleichtert.